Ievads attēlu apstrādē

Praktiskais darbs Nr. X

Students: Renārs Liepa

Studenta apliecības Nr.: 251RDB061

Saite uz programmēšanas kodu: https://github.com/liepinjjjsss/lab_d_attelu_apstrade/blob/main/imageprocessing.py

Šo veidni var iekopēt sev uz diska (Save a copy in Drive), lai pēc tam to varētu atbilstoši aizpildīt un pārveidot uz PDF.

Programmas kods ar izvadi

Darbā jābūt iekļautam palaistam Python kodam, t. i., kodam jābūt izpildītam tā, lai dokumentā būtu redzamas visas vizualizācijas, attēli, grafiki un cita programmas izvade. Nepalaists kods vai kods bez redzamiem rezultātiem netiks uzskatīts par korekti iesniegtu. Kodam jābūt bez komentāriem.

```
from PIL import Image
from IPython.display import display
from google.colab import files
import numpy as np
import math
import os

uploaded = files.upload()
filenames = list(uploaded.keys())

os.makedirs("result_images", exist_ok=True)
img_1 = Image.open(filenames[0]).convert("RGB")
img_2 = Image.open(filenames[1]).convert("RGB")

pixels_1 = np.array(img_1)
pixels_2 = np.array(img_2)

height_1, width_1, channel_1 = pixels_1.shape
height_2, width_2, channel_2 = pixels_2.shape

result_height = 512
result_width = 512
result_channel = 3

result_pixels = np.zeros((result_height, result_width, result_channe
```

```
result_pixels = np.zeros((result_height, result_width, result_channe

opacity_coefficient = 0.7
for y in range(height_1):
    for x in range(width_1):
        r1, g1, b1 = pixels_1[y, x]
        r2, g2, b2 = pixels_2[y, x]

        r = opacity_coefficient*r1 + (1-opacity_coefficient)*r2
        g = opacity_coefficient*g1 + (1-opacity_coefficient)*g2
        b = opacity_coefficient*b1 + (1-opacity_coefficient)*b2

        result_pixels[y, x] = r, g, b

img = Image.fromarray(result_pixels)
print("\n OPACITY")
display(img)


for y in range(height_1):
    for x in range(width_1):
        r1, g1, b1 = pixels_1[y, x]
        r2, g2, b2 = pixels_2[y, x]

        r = abs(int(r1)-int(r2))
        g = abs(int(g1)-int(g2))
        b = abs(int(b1)-int(b2))

        result_pixels[y, x] = r, g, b

img2 = Image.fromarray(result_pixels)
print("\n DIFFERENCE")
display(img2)


for y in range(height_1):
    for x in range(width_1):
        r1, g1, b1 = pixels_1[y, x]
        r2, g2, b2 = pixels_2[y, x]

        r1_f, r2_f = r1 / 255.0, r2 / 255.0
        g1_f, g2_f = g1 / 255.0, g2 / 255.0
        b1_f, b2_f = b1 / 255.0, b2 / 255.0

        if r1_f <= 0.5:
            r = (2*r1_f-1)*(r2_f-r2_f*r2_f)+r2_f
        else:
            r = (2*r1_f-1)*(math.sqrt(r2_f)-r2_f)+r2_f

        if g1_f <= 0.5:
            g = (2*g1_f-1)*(g2_f-g2_f*g2_f)+g2_f
        else:
            g = (2*g1_f-1)*(math.sqrt(g2_f)-g2_f)+g2_f

        if b1_f <= 0.5:
```

```
        if b1_f <= 0.5:
            b = (2*b1_f-1)*(b2_f-b2_f*b2_f)+b2_f
        else:
            b = (2*b1_f-1)*(math.sqrt(b2_f)-b2_f)+b2_f


        result_pixels[y, x] = r*255, g*255, b*255

img3 = Image.fromarray(result_pixels)
print("\n SOFT LIGHT")
display(img3)



for y in range(height_1):
    for x in range(width_1):
        r1, g1, b1 = pixels_1[y, x]
        r2, g2, b2 = pixels_2[y, x]

        r1_f, r2_f = r1 / 255.0, r2 / 255.0
        g1_f, g2_f = g1 / 255.0, g2 / 255.0
        b1_f, b2_f = b1 / 255.0, b2 / 255.0

        if r1_f <= 0.5:
            r = 2*r1_f*r2_f
        else:
            r = 1 - 2*(1-r1_f)*(1-r2_f)

        if g1_f <= 0.5:
            g = 2*g1_f*g2_f
        else:
            g = 1 - 2*(1-g1_f)*(1-g2_f)

        if b1_f <= 0.5:
            b = 2*b1_f*b2_f
        else:
            b = 1 - 2*(1-b1_f)*(1-b2_f)

        result_pixels[y, x] = r*255, g*255, b*255

img4 = Image.fromarray(result_pixels)
print("\n HARD LIGHT")
display(img4)
```

Browse… 2 files selected.

Grey-wolf.jpg(image/jpeg) - 82731 bytes, last modified: n/a - 100% done
starry-night-sky-stockcake.jpg(image/jpeg) - 75440 bytes, last modified: n/a - 100% done

```
Saving Grey-wolf.jpg to Grey-wolf (5).jpg
Saving starry-night-sky-stockcake.jpg to starry-night-sky-stockcake
```

```
 OPACITY
```

DIFFERENCE

SOFT LIGHT

SOFT LIGHT

HARD LIGHT

Secinājumi

Satur sakarīgus un pamatotus secinājumus par izveidotu darbu un eksperimentiem. Var vadīties pēc uzdevumā dotām vadlīnijām. Secinājumus jāraksta patstāvīgi, MI rīku pielietošana nav pieņemama.

Papildus kods formatēta HTML faila iegūšanai

Ar šo koda daļu ir iespējams saglabāt Colab Notebook kā HTML, kuru pēc tam var "izdrukāt" kā PDF, kopā ar visām izvadēm. Šī pieeja nodrošina labāku programmas koda un izvades attēlu renderēšanu, nekā mēģinājumi to drukāt pa taisno no pārlūka.

```
#Šo var palaist tad, ja Google Colab sūdzās, ka nav pieinstālēts nb
!pip install nbconvert
```

```
#Šis ir nepieciešams, lai dotu Google Colab pieeju diskam, kurā gla
from google.colab import drive
drive.mount('/content/drive')
```

Show hidden output

Next steps: Explain error

```
# Šeit norādiet ceļu pie Jūsu Colab notebook faila, kur tieši tas a
!jupyter nbconvert "/content/drive/MyDrive/Colab Notebooks/Praktisk
  --to html \
  --embed-images
```

```
#Notiek HTML faila lejuplāde
from google.colab import files
files.download("/content/drive/MyDrive/Colab Notebooks/Praktiskais d
```

```
---------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent
call last)
/tmp/ipython-input-3167265266.py in <cell line: 0>()
      1 #Notiek HTML faila lejuplāde
      2 from google.colab import files
----> 3 files.download("/content/drive/MyDrive/Colab Notebooks/
Praktiskais darbs X.html")

/usr/local/lib/python3.12/dist-packages/google/colab/files.py in
download(filename)
```

```
download(filename)
    228   if not _os.path.exists(filename):
    229     msg = 'Cannot find file: {}'.format(filename)
--> 230     raise FileNotFoundError(msg)  # pylint:
disable=undefined-variable
    231
    232   comm_manager = _IPython.get_ipython().kernel.comm_manager
```

Next steps: Explain error

Tālāk atverot lejuplādēto HTML failu pārlūkā, to var izdrukāt kā PDF (Save to PDF).